HARD-2：动态内存分配

前置知识

1.堆：**堆**是进程虚拟地址空间中的一段可以动态扩展和收缩的内存区域。

2.**动态内存分配器**：管理动态内存（分配，释放，分割，合并等问题）。**空闲块**：堆中未被使用，可以被分配出去的块。**对齐**：要求内存地址是某个数的整数倍（内部空间可能无法完全利用）。**碎片**：无法被有效利用的内存：包括已分配给某个块但未被实际使用的空间，以及外部无法构成足够大空闲块。

3.**隐式**：不额外维护"空闲链表块"，每个块的头部记录信息，分配器遍历所有块操作。**显式**：在空闲块中间维护一个链表（通常是双向链表），指针存放在空闲块的有效载荷区域，分配器遍历空闲块链表，而不是整个堆块。

4.**吞吐量**单位时间内能完成的malloc/free操作，衡量效率（关键在于时间长短）。影响吞吐量的因素有搜索空闲块的速度、是否进行合并、是否分割等。**空间利用率**是（已分配块的有效载荷总需求/堆的总大小），其中碎片和元数据都会降低空间利用率。而分配器的目标之一是在可接受的**吞吐量**下，尽可能提高**空间利用率**。

实验0

思考：`sbrk()`函数移动堆顶是直上直下的，因此，正如题目所说，必须堆顶没有活跃对象才能free，并不能对中间的某部分进行操作，所以不能free新增长的空间。

实验一

```
PUT(heap_listp, 0);
PUT(heap_listp + 1*WSIZE, PACK(8, 1));
PUT(heap_listp + 2*WSIZE, PACK(8, 1));
PUT(heap_listp, PACK(0, 1));
```

回答一下学长的"巧思"：其实无论第一行和第四行是什么，在这个隐式空闲链表中都不会对分配器的操作有任何影响，因为第一块作为填充读不到也没意义，最后一块更是直接在`mm_init`中被覆盖了，但这样写代码逻辑总是不太舒服。

元信息：管理字段。此处是记录每个块的大小，以及是否空闲的信息。

思考：1.为什么用链表不用数组？我觉得数组想在中间插入/删除元素难以做到，但链表在我们写实验体的时候就发现，无论是释放，调用，还是分割空闲块，指针所指的地方会根据宏的运算方式直接指向下一个块，不需要我们手动维护，非常方便。2.ftr主要作用于块向前扫描，因为我们不知道前一个快有多大，所以暂时找不到指向前一个块的指针，而ftr紧挨着当前块的header就方便查询前一块信息。**我觉得，这个所谓的链表，更应该看作相邻的一种物理关系**

思考：1.为什么用宏定义？代码可读性好，避免冗余代码。2.把size和alloc打包可以减少空间的使用，不打包为了对齐就又要专门留四个字节给alloc，浪费空间。3.为什么可以得到上一块地址？回退八个字节刚好啊，ftr就紧挨着hdr，各占4字节。4.对齐的作用：根据查阅的资料：***`malloc`返回的指针必须满足最大对齐要求（通常 8 或 16 字节），否则调用者把返回地址当`double`、指针数组、SIMD 类型用时，在要求对齐的架构上会出错或性能下降。***（这好像是硬件要求，不是很了解）

下面是运行截图的一部分。

```
Address: 0x58f0e5111010
Size: 8008
Allocated: 1

Address: 0x58f0e5112f58
Size: 0
Allocated: 1
========================= mm_malloc ==========================

========================= mm_malloc ==========================
Address: 0x58f0e5110008
Size: 8
Allocated: 1

Address: 0x58f0e5110010
Size: 208
Allocated: 1

Address: 0x58f0e51100e0
Size: 64
Allocated: 1

Address: 0x58f0e5110120
Size: 3824
Allocated: 0

Address: 0x58f0e5111010
Size: 8008
Allocated: 1

Address: 0x58f0e5112f58
Size: 0
Allocated: 1
========================= mm_malloc ==========================

========================= mm_free ============================
Address: 0x58f0e5110008
Size: 8
Allocated: 1

Address: 0x58f0e5110010
Size: 208
Allocated: 0

Address: 0x58f0e51100e0
Size: 64
```

一些在linux下c语言的调试尝试，还不是很熟练。这一堆东西验证了1.size转化为asize，2.空闲的3888（0xf30）字节经过place后减去64变成3824字节。

```
Breakpoint 1, mm_malloc (size=50) at HARD-2.c:131
131         if(size==0){return NULL;}
(gdb) c
Continuing.

Breakpoint 2, place (bp=0x5555555590e0 "", asize=64) at HARD-2.c:107
```

```
(gdb) x/xw (char*)bp-4
0x5555555590dc: 0x00000f30
(gdb) ni
0x000055555555547b      107         size_t size = GET_SIZE(HDRP(bp));
```

```
(gdb) print GET_SIZE(HDRP(NEXT_BLKP(bp)))
$7 = 3824
(gdb) c
Continuing.

========================== mm_malloc ==========================
Address: 0x555555559008
Size: 8
Allocated: 1

Address: 0x555555559010
Size: 208
Allocated: 1

Address: 0x5555555590e0
Size: 64
Allocated: 1

Address: 0x555555559120
Size: 3824
Allocated: 0
```

内部碎片处理：感觉主要的处理逻辑是控制空间浪费（减少内部碎片产生）比如place中会检查size和asize的大小关系，如果剩余空间大于MINBLOCK，则分配出去。不会产生多余的hdr和ftr。

隐式分配器的时间复杂度最坏是O(N)，因为是链表，所以当需要遍历整个链表时时间就是O(N)。为什么要遍历整个链表？因为隐式并不知道空闲块在哪，只能根据链表读每一块的信息。如何优化？看看能不能只把空闲块穿起来，就可以降低时间复杂度了。

为什么不在搜索不到足够大的空闲块时遍历合并？我认为延迟是很重要的原因，全堆遍历直接把原先的时间复杂度翻倍了，并且合并后说不定空闲块的大小还是不够。成本又高又不一定有用。

相对可行的方法：之前每次free后检查两边就很好，毕竟时间复杂度是O(1)。或者感觉可以在`find_fit`中搜索到空闲块，但不够大时查前后块是否空闲，空闲就合并到一起，这样就不用再遍历一次链表。

实验二

只写了一部分，燃尽了